Deep Learning

Assignment #3

Recurrent Neural Networks

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 15 points*
- *Timeline: Assignment # 3 available 29th Oct, Code submission deadline 12th Nov, offline defense deadline till 19th Nov*

While completing this assignment, you will develop and strengthen the following skills:

- *Approaching a real-world machine learning problem with given data and a target metric.*
- *Conducting effective Exploratory Data Analysis (EDA).*
- *Designing a reliable validation strategy and monitoring it throughout the entire development process.*
- *Building a RNN-based model from scratch or using existing architectures.*
- *Performing health checks on the optimization process.*
- *Using Transfer Learning techniques and other advanced Deep Learning techniques*
- *Utilizing specialized Deep Learning frameworks and logging tools.*
- *Creating a comprehensive performance report.*

Task

Task Selection:

- *Take part in the dedicated Kaggle In-Class competition*

EDA and Metric Analysis:

- *Perform Exploratory Data Analysis (EDA).*
- *Analyze the proposed target metric: discuss its strengths, weaknesses, and potential edge cases.*
- *Suggest complementary metrics to capture other aspects of performance.*
- *Propose an alternative target metric, with justification.*

Validation:

- *Design a validation strategy based on common sense and expected correlation with Public leaderboard scores.*
- *Explain your motivation for the chosen strategy.*
- *Discuss possible reasons for low or high correlation with leaderboard results.*
- *Perform adversarial validation if necessary (make sure to do it with the same model as the main task)*

GRU/LSTM model:

- *Set up experiment and logging infrastructure using high-level libraries (like Pytorch Lightning and W&B)*
- *Use existing architecture based on GRU/LSTM to solve Named Entity Recognition task. Use ONLY recurrent-like models in this assignment. Although using transformer encoder models like e.g. BERT may give you slightly better results - using transformers or other self-attention models is out of scope in this assignment.*
- *Experiment with different Optimizers, Schedulers, hyperparameters, and all other elements, which were covered in the Lecture materials*
- *Try different tokenization algorithms: e.g. word-level, BPE*
- *Try using more recurrent layers or bidirectional modification of GRU/LSTM*
- *Try pre-trained multilingual embeddings e.g. fasttext or glove or use pre-trained weights as initialized embedding during model finetuning.*
- *Try threshold optimization and different entity-merging algorithms.*
- *Collect all relevant results: loss/metric curves, gradient flow visualizations, parameter tuning logs, etc.*
- *Submit predictions and compare Public leaderboard metrics with local validation results.*
- *Conclude whether the model is adequate.*

Ensembling:

- *Combine several GRU/LSTM models using the ensembling (NOT stacking) technique*
- *[Optionally] Use Logistic Regression in order to perform stacking*
- *Submit predictions to obtain Public leaderboard metrics, and compare them with local validation results.*

Final Report

- *Prepare a comprehensive report including:*
  - *data insights,*
  - *feature engineering,*
  - *tried models,*
  - *chosen hyperparameters,*
  - *metrics and optimization curves,*
  - *model insights and conclusions.*
- *Reports may be prepared in LaTeX or Markdown (README.md).*

NOTES

- *Use ONLY recurrent-like models in this assignment. Although using transformer encoder models like e.g. BERT may give you slightly better results - using transformers or other self-attention models is out of scope in this assignment.*
- *Do not hesitate to include failed ideas or experiments in your report. Documenting unsuccessful attempts is a strong indication that you systematically explored different approaches and gradually arrived at the best-performing solution, rather than relying on a "lucky shot." Moreover, analyzing failed approaches can often lead to valuable insights about the dataset, the task itself, and the behavior of deep learning algorithms. Such reflections may also highlight potential directions for future improvements and demonstrate a deeper understanding of the problem-solving process.*
- *Keep your EDA concise. Avoid overloading it with excessive plots, histograms, or tables. Focus only on key aspects of the data (e.g., missing values, target distribution). If a part of your EDA does not lead to meaningful conclusions, you likely don't need it (for example, full pairwise correlation matrices). Do NOT follow the typical Kaggle-style EDA — most of them are far too verbose and unfocused.*
- *Prepare your presentation carefully. A messy speech with too much code or too many files makes it nearly impossible for the audience to follow. Think in advance about what you want to highlight*
- *Using Keras, TensorFlow, or Chainer is a RED FLAG and will result in an instant BAN!*
- *You should thoroughly analyze the training/test split and consider time and group factors in your validation. You should also use the same splits for different ML/DL approaches so that the results can be accurately compared. And if there is a huge mismatch between CV vs kaggle LB - consider using adversarial validation.*
- *Your final report MUST include both CV and LB scores for all experiments.*

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	***Points***
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	*6*
Practical question about the submitted code during defense	*4*
Theoretical part during defense (2 questions + 1 exercise)	*5*
Criterion	***Points***

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the Lecturer will evaluate each student's individual contribution, demonstrated skills, and depth of knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 2 and 1 Exercise from the Appendix 3, but will not necessarily be restricted to them.

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- *Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.*
- *Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.*
- *Language assistance tools: Grammarly, DeepL, Google Translate, etc.*
- *Local LLMs or self-hosted AI models.*

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- *Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.*
- *Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.*
- *Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.*
- *Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.*
- *Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.*
- *Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.*

Conditions for Use

Full Transparency:

- *Any use of AI tools must be disclosed in your submission.*
- *Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.*
- *If using online tools, include chat links (if available) or full transcripts in an appendix of your report.*

Partial Contribution Only:

- *AI tools should support your work — they must not produce the complete assignment, lab, or report.*
- *You must be able to explain and defend any AI-generated content during oral defense or Q&A*

Examples

Acceptable Uses:

- *“Correct grammar and style in this paragraph of my report.”*
- *“Write a function to load images from a directory and resize them.”*
- *“Suggest 3 ideas for improving model generalization.”*

Unacceptable Uses:

- *“Write the entire peer review for this paper.”*
- *“Generate the complete solution code for this assignment task.”*

Appendix 1 (Theoretical Questions)

- *Write the forward equations of a vanilla RNN and define all symbols.*
- *Why does sharing the Wh matrix in RNN across time reduce the number of learnable parameters?*
- *Give conditions under which a vanilla RNN is likely to suffer from exploding gradients.*
- *Provide a short argument for vanishing/exploding gradients for vanilla RNN.*
- *What’s the role of embeddings in sequence models?*
- *Explain padding sequences in PyTorch and why they matter for variable-length batches.*
- *When might you prefer bidirectional over unidirectional RNNs? Give concrete tasks.*
- *Contrast LSTM vs. GRU in terms of parameter count and expressive power (one pro/one con each).*
- *Provide a brief argument for why tanh vs. ReLU inside LSTM/GRU can change stability.*
- *In bidirectional LSTM, how do you combine forward/backward states for classification?*
- *Describe a simple strategy to tune decision thresholds when optimizing F1 for sequence classification.*
- *For sequence classification, compare using the last hidden state vs. pooling of per-time-step projections*

Appendix 2 (Practical Exercises)

1. **Vanilla RNN (forward pass):** A vanilla RNN with ReLU activation is defined as

$$h_t = \mathrm{ReLU}(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad \hat{y}_t = W_{hy}h_t + b_y.$$

Let hidden size $= 2$, sequence length $= 2$, and

$$x_1 = \begin{bmatrix}1\\1\end{bmatrix}, \quad x_2 = \begin{bmatrix}-1\\1\end{bmatrix}, \quad W_{xh} = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}, \; W_{hh} = \begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}, \; b_h = \begin{bmatrix}-3\\3\end{bmatrix}, \; h_0 = \begin{bmatrix}0\\0\end{bmatrix}.$$

   (a) Compute h_1 and h_2 exactly (no decimals). Use $\mathrm{ReLU}(t) = \max(0, t)$ elementwise.

2. **Vanilla RNN with tanh: gradient propagation.** Definition: $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$.

   (a) Calculate the Jacobian $\dfrac{\partial h_{t+s}}{\partial h_t}$

   (b) Assume $W_{hh} = aI_2$ with $a = 0.9$, and along a given sequence the pre-activations at the hidden layer satisfy $W_{xh}x_k + W_{hh}h_{k-1} + b_h = \mathbf{0}$ for $k = t+1, \ldots, t+s$ (so that $\tanh'(\cdot) = 1$ on those steps). Show that the Jacobian $\dfrac{\partial h_{t+s}}{\partial h_t} = (aI_2)^s$

   (c) Briefly discuss what happens if the pre-activations are not 0 so that $\tanh'$ is < 1.

3. **Vanilla RNN with sigmoid: gradient upper bound.** Definition: $h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$ with $\sigma(u) = \frac{1}{1+e^{-u}}$ applied elementwise. Assume $\|W_{hh}\|_2 = 1$. **Spectral norm (operator 2-norm):** For any matrix M,

$$\|M\|_2 \;=\; \sup_{\|v\|_2=1} \|Mv\|_2 \,.$$

Recall that for any u, $\sigma'(u) = \sigma(u)\big(1 - \sigma(u)\big)$.

   (a) Show that an upper bound for the gradient operator norm is

$$\left\| \frac{\partial h_{t+s}}{\partial h_t} \right\|_2 \;\le\; \big(\|W_{hh}\|_2 \cdot \sup_u \sigma'(u)\big)^s \;\le\; \big(\tfrac{1}{4}\big)^s \,.$$

   (b) Interpret these numbers: what do they imply about the maximum possible gradient magnitude through s steps?

4. **LSTM: cell-state dynamics and vanishing.** LSTM (per coordinate) is defined by

$$\begin{aligned} i_t &= \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i), \quad & f_t &= \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f),\\ o_t &= \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o), \quad & g_t &= \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g),\\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

   (a) Show that the Jacobian of the cell state satisfies

$$\frac{\partial c_{t+s}}{\partial c_t} = \prod_{k=1}^{s} \mathrm{Diag}(f_{t+k})$$

   (diagonal product elementwise).

   (b) Discuss vanishing: how does the product of forget gates govern gradient flow in c?